篤學力行
守正求新

杭州电子科技大学

实验一　开关量输入输出实验

课　程　微机原理与接口技术

学　院　自动化学院

专　业　自动化 (新工科试验班)

学　号　24061428

学生姓名　张跃哲

指导教师　金朝阳

完成日期　2026 年 4 月 17 日

目录

1　实验要求与实验目的　1
1.1　实验要求 1
1.2　实验目的 1
1.3　设计提示 1

2　Proteus 原理图与硬件设计说明　1
2.1　原理图截图 2
2.2　电路组成与功能对应关系 2
2.3　主要元件清单 3

3　软件设计与流程分析　3
3.1　程序流程图 4
3.2　完整实验代码 4
3.3　关键代码逐段说明 5

4　调试过程与实验结果　5
4.1　调试过程记录 5
4.2　结果验证表 6
4.3　编译信息 6

5　问题分析与改进建议　6

6　实验结论　6

A　附录：编译清单摘要　6

1　实验要求与实验目的

1.1　实验要求

根据实验指导书，实验一的要求如下：

1. 利用 AT89C51 微控制器的 P0 口作开关量输入口，P1 口作开关量输出口。
2. 当 P0.x 端开关闭合时，对应 P1.x 口的 LED 发光二极管点亮；当 P0.x 端开关断开时，对应 P1.x 口的 LED 发光二极管熄灭。
3. 画出 AT89C51 实现上述功能的完整电路图，包括微控制器电源、复位电路、晶振电路和控制电路。
4. 完成全部程序和电路调试工作。

1.2　实验目的

1. 掌握 AT89C51 微控制器最基本电路的设计方法。
2. 了解微控制器 I/O 端口的使用方法。

1.3　设计提示

P0 口作为 I/O 使用时，需要配置上拉电阻，避免端口悬空导致读值不稳定。

2　Proteus 原理图与硬件设计说明

2.1 原理图截图

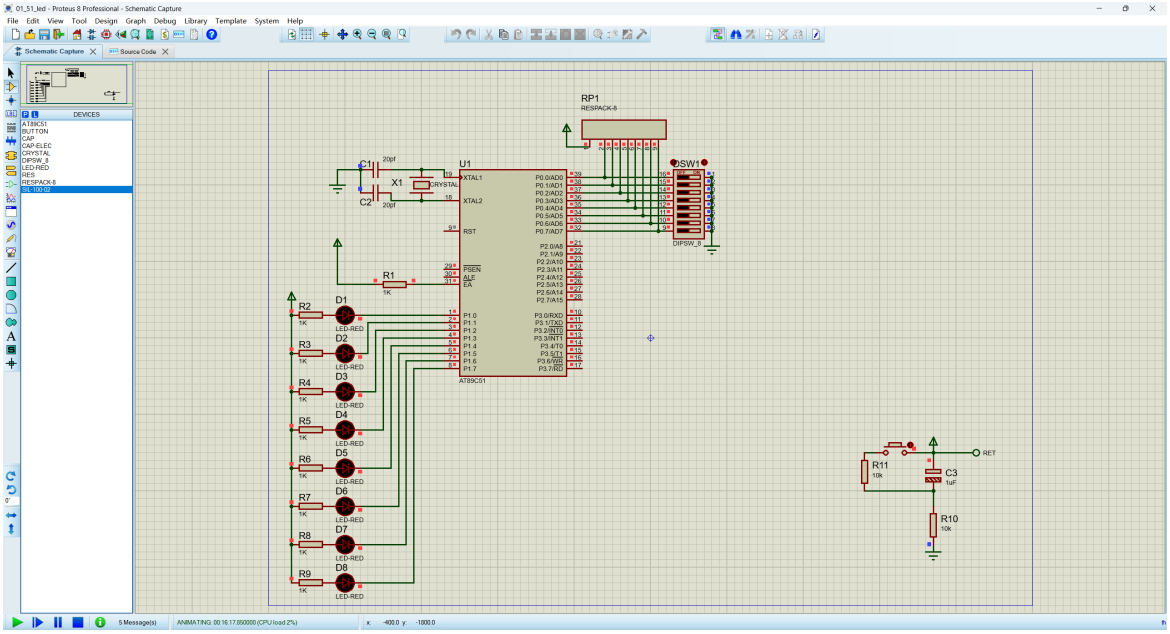

图 1　实验一 Proteus 原理图截图

2.2 电路组成与功能对应关系

本实验硬件电路由四个部分组成：

- 电源与基础引脚：为 AT89C51 提供稳定工作电源，并完成必要控制引脚配置。
- 复位电路：保证上电时单片机进入确定初始状态，程序从复位入口开始执行。
- 晶振电路：由晶振和电容构成时钟网络，为 CPU 提供系统时钟。
- I/O 控制电路：P0 连接拨码开关输入，P1 连接 LED 输出，完成开关状态到灯状态的位对应映射。

  结合原理图可见：

1. SW1（DIPSW_8）提供 8 位开关量输入；
2. P0.0–P0.7 接收上述输入信号；
3. P1.0–P1.7 分别驱动 D1–D8；
4. 每个 LED 串联电阻，起限流与保护作用。

2.3 主要元件清单

表 1　实验主要元件与模型

序号	元件名称	元件规格	所在元件库	工具模型
1	微控制器	AT89C51	Microprocessor	Component mode
2	按钮/开关	BUTTON	Switches & Relay	Component mode
3	晶振	CRYSTAL	Miscellaneous	Component mode
4	发光二极管	LED-RED	Optoelectronics	Component mode
5	电容	CAP / CAP-ELEC	Capacitors	Component mode
6	电阻	RES	Resistors	Component mode
7	拨码开关	DIPSW_8	Switches & Relay	Component mode
8	电源与地	POWER / GROUND	Terminals	Terminals mode
9	电源输入端	SIL-100-02	Connectors	Component mode

3 软件设计与流程分析

3.1 程序流程图

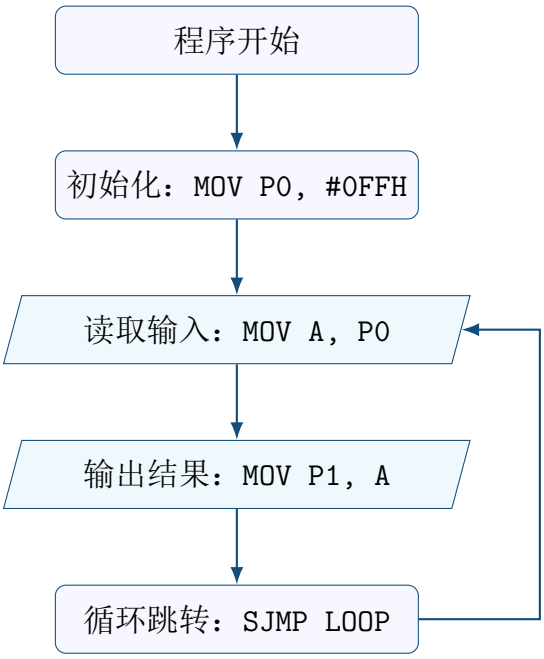

图 2　实验一程序流程图

3.2 完整实验代码

Listing 1　01_single_led.A51 完整源码

```
ORG 0000H
LJMP MAIN

ORG 0030H
MAIN:
LOOP:
    MOV P0, #0FFH
    MOV A, P0
    MOV P1, A
    SJMP LOOP
END
```

3.3 关键代码逐段说明

表 2　关键指令功能说明

指令或语句	作用说明
`ORG 0000H` + `LJMP MAIN`	定义复位入口，单片机上电后从 0000H 跳转到主程序地址。
`ORG 0030H`	主程序起始地址定义，便于与中断向量区域区分。
`MOV P0, #0FFH`	将 P0 各位写 1，使端口处于读取外部开关状态的工作方式。
`MOV A, P0`	将 P0 当前 8 位输入状态读入累加器 A。
`MOV P1, A`	将输入值直接输出到 P1，实现 8 位一一对应控制。
`SJMP LOOP`	构成无限循环，实现实时轮询与实时响应。

4 调试过程与实验结果

4.1 调试过程记录

1. 在 Keil 中编译 `01_single_led.A51`，确认语法与目标文件生成正常。
2. 在 Proteus 中连接 AT89C51、拨码开关、LED、晶振、复位和供电电路。
3. 加载编译生成程序，启动仿真。
4. 逐位拨动 SW1，观察 D1–D8 是否按位同步变化。
5. 对全灭、全亮、奇偶位组合等典型输入进行对照验证。

4.2 结果验证表

表 3　输入输出对应关系验证

测试序号	P0 输入（开关状态）	预期 P1/LED 输出	结果
1	0x00	全灭	符合
2	0xFF	全亮	符合
3	0x55	奇数位亮、偶数位灭	符合
4	0xAA	偶数位亮、奇数位灭	符合
5	任意组合	位对位同步显示	符合

4.3 编译信息

从 A51 编译列表可确认：

- 汇编器版本：A51 V8.2.7.0；
- 关键符号地址：`MAIN=0030H`，`LOOP=0030H`，`P0=0080H`，`P1=0090H`；
- 编译结果：**0 warning，0 error**。

5 问题分析与改进建议

1. P0 口使用时应确认上拉网络有效，避免输入悬空导致 LED 状态抖动。
2. 实际硬件中可加入按键消抖或软件延时，提高状态显示稳定性。
3. 可扩展为中断方式采样，减少无效轮询占用。

6 实验结论

本实验完成了基于 AT89C51 的开关量输入输出控制：P0 口读取开关状态，P1 口输出 LED 控制信号，最终实现输入与输出一一对应。通过本次实验，验证了单片机最基本电路（电源、复位、晶振、I/O 控制）与端口编程方法，达到实验指导书要求。

A 附录：编译清单摘要

Listing 2　01_single_led.LST（节选）

```
A51 MACRO ASSEMBLER  01_SINGLE_LED                                                    04/17/2026
   01:21:37 PAGE     1


MACRO ASSEMBLER A51 V8.2.7.0
OBJECT MODULE PLACED IN 01_single_led.OBJ
ASSEMBLER INVOKED BY: C:\Keil\C51\BIN\A51.exe E:\homework\51_proteus
   \01_single_led\01_single_led.A51 DEBUG SYMBOLS OBJEC
                      T(01_single_led.OBJ)

LOC  OBJ            LINE     SOURCE

0000                   1     ORG 0000H
0000 020030            2     LJMP MAIN
                       3
0030                   4     ORG 0030H
0030                   5     MAIN:
0030                   6     LOOP:
0030 7580FF            7         MOV P0, #0FFH
0033 E580              8         MOV A, P0
0035 F590              9         MOV P1, A
0037 80F7             10         SJMP LOOP
                      11     END
```

```
A51 MACRO ASSEMBLER  01_SINGLE_LED                                                    04/17/2026
   01:21:37 PAGE     2

SYMBOL TABLE LISTING
------ ----- -------


N A M E             T Y P E  V A L U E   ATTRIBUTES

LOOP . . . . . . .  C ADDR   0030H   A
MAIN . . . . . . .  C ADDR   0030H   A
P0 . . . . . . . .  D ADDR   0080H   A
P1 . . . . . . . .  D ADDR   0090H   A

```

```

REGISTER BANK(S) USED: 0


ASSEMBLY COMPLETE.  0 WARNING(S), 0 ERROR(S)
```